NAME: Deepti

CUID :24250168

ROLL NO.16 SEC: B

Experiment No. 10

Character, Digit, or Face Classification using Convolutional Neural Networks

```
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf

from tensorflow.keras import layers, models
from tensorflow.keras.datasets import mnist
from sklearn.metrics import confusion_matrix, classification_report

# Load the MNIST handwritten digit dataset
(x_train, y_train), (x_test, y_test) = mnist.load_data()

# Normalize pixel values from 0-255 to 0-1
x_train = x_train.astype("float32") / 255.0
x_test = x_test.astype("float32") / 255.0

# Add the channel dimension
x_train = np.expand_dims(x_train, axis=-1)
x_test = np.expand_dims(x_test, axis=-1)

# Create a validation set from the training data
x_validation = x_train[-5000:]
```

```
y_validation = y_train[-5000:]

x_train = x_train[:-5000]
y_train = y_train[:-5000]

# Display one sample image
plt.imshow(x_train[0].squeeze(), cmap="gray")
plt.title(f"Sample Digit: {y_train[0]}")
plt.axis("off")
plt.show()

# Create the CNN model
model = models.Sequential([
    layers.Input(shape=(28, 28, 1)),

    # First convolution layer
    layers.Conv2D(32, (3, 3), activation="relu"),

    # First pooling layer
    layers.MaxPooling2D((2, 2)),

    # Second convolution layer
    layers.Conv2D(64, (3, 3), activation="relu"),

    # Second pooling layer
    layers.MaxPooling2D((2, 2)),

    # Convert feature maps into a vector
```

```
    layers.Flatten(),

    # Fully connected layer
    layers.Dense(64, activation="relu"),

    # Output layer for 10 digits
    layers.Dense(10, activation="softmax")
])
# Display model architecture
model.summary()

# Compile the model
model.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)

# Train the CNN
history = model.fit(
    x_train,
    y_train,
    validation_data=(x_validation, y_validation),
    epochs=5,
    batch_size=64
)

# Evaluate the model on the test dataset
```

```
test_loss, test_accuracy = model.evaluate(
    x_test,
    y_test,
    verbose=0
)

print(f"Test Accuracy: {test_accuracy * 100:.2f}%")
print(f"Test Loss: {test_loss:.4f}")

# Plot training and validation accuracy
plt.plot(history.history["accuracy"], label="Training Accuracy")
plt.plot(history.history["val_accuracy"], label="Validation Accuracy")
plt.title("Training and Validation Accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend()
plt.show()

# Plot training and validation loss
plt.plot(history.history["loss"], label="Training Loss")
plt.plot(history.history["val_loss"], label="Validation Loss")
plt.title("Training and Validation Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend()
plt.show()

# Predict labels for the test dataset
```

```
predictions = model.predict(x_test, verbose=0)
predicted_labels = np.argmax(predictions, axis=1)

# Display confusion matrix
cm = confusion_matrix(y_test, predicted_labels)
plt.figure(figsize=(8, 6))
plt.imshow(cm, cmap="Blues")
plt.title("Confusion Matrix")
plt.xlabel("Predicted Label")
plt.ylabel("Actual Label")
plt.colorbar()
plt.show()
# Print classification report
print("Classification Report")
print(classification_report(y_test, predicted_labels))

# Display some predictions
plt.figure(figsize=(10, 5))
for i in range(10):
    plt.subplot(2, 5, i + 1)
    plt.imshow(x_test[i].squeeze(), cmap="gray")
    plt.title(
        f"Actual: {y_test[i]}\nPredicted: {predicted_labels[i]}"
    )
    plt.axis("off")
plt.tight_layout()
plt.show()
# Save the trained model
```

```
model.save("mnist_cnn_model.keras")
print("Model saved as mnist_cnn_model.keras")

# Observation
# The CNN automatically learns useful image features from handwritten digits.
# Convolutional layers detect edges and shapes.
# Pooling layers reduce spatial size and computation.
# Dense layers perform final classification.
# The confusion matrix shows correctly and incorrectly classified digits.

# Question 1: What is image classification?
# Image classification assigns a label to an entire image.
# Object detection identifies objects with bounding boxes, while segmentation labels
# individual pixels or regions.

# Question 2: Explain the architecture and working principle of a CNN.
# A CNN uses convolutional layers to extract features, pooling layers to reduce dimensions,
# and fully connected layers to perform classification. The output layer produces class
# probabilities.

# Question 3: Explain convolution, pooling, and fully connected layers.
# Convolutional layers learn visual features such as edges and shapes.
# Pooling layers reduce feature-map dimensions and improve computational efficiency.
# Fully connected layers combine learned features for final classification.

# Question 4: Why is normalization performed?
# Normalization scales pixel values to a smaller range, usually 0 to 1.
# It improves numerical stability and helps the model train faster.
```

Question 5: Explain ReLU and Softmax.

ReLU introduces non-linearity by replacing negative values with zero.

Softmax converts output values into probabilities whose sum is one.

Question 6: What is a confusion matrix?

A confusion matrix compares actual labels with predicted labels.

It shows correct predictions and the types of classification errors.

Question 7: Differentiate Accuracy, Precision, Recall, and F1-Score.

Accuracy is the percentage of all correct predictions.

Precision is the percentage of predicted positive samples that are actually positive.

Recall is the percentage of actual positive samples correctly identified.

F1-Score is the harmonic mean of precision and recall.

Question 8: Compare SIFT/HOG + SVM with CNN classification.

SIFT/HOG + SVM uses manually designed features and a separate classifier.

CNNs learn features and classification jointly from training data.

CNNs often provide better performance but require more data and computation.

Question 9: Mention five real-world applications.

Applications include handwritten digit recognition, face recognition, medical image

classification, biometric authentication, and document analysis.

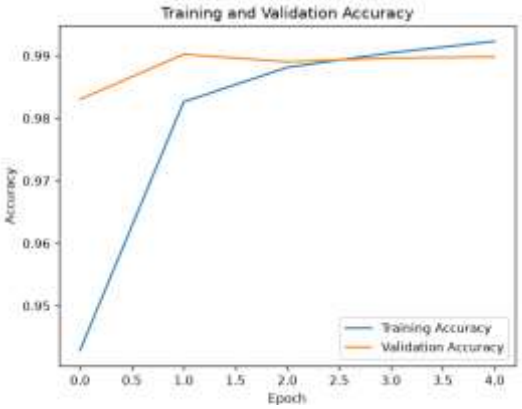
Question 10: How do augmentation, transfer learning, and tuning improve performance?

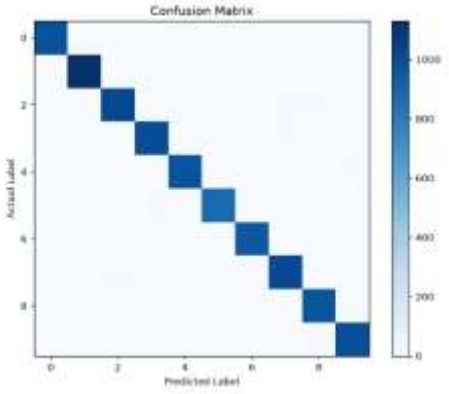
Data augmentation creates varied training examples and reduces overfitting.

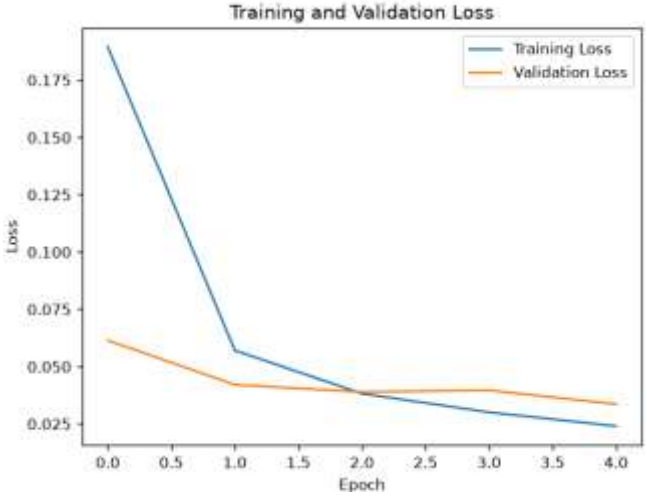
Transfer learning uses knowledge from an existing model.

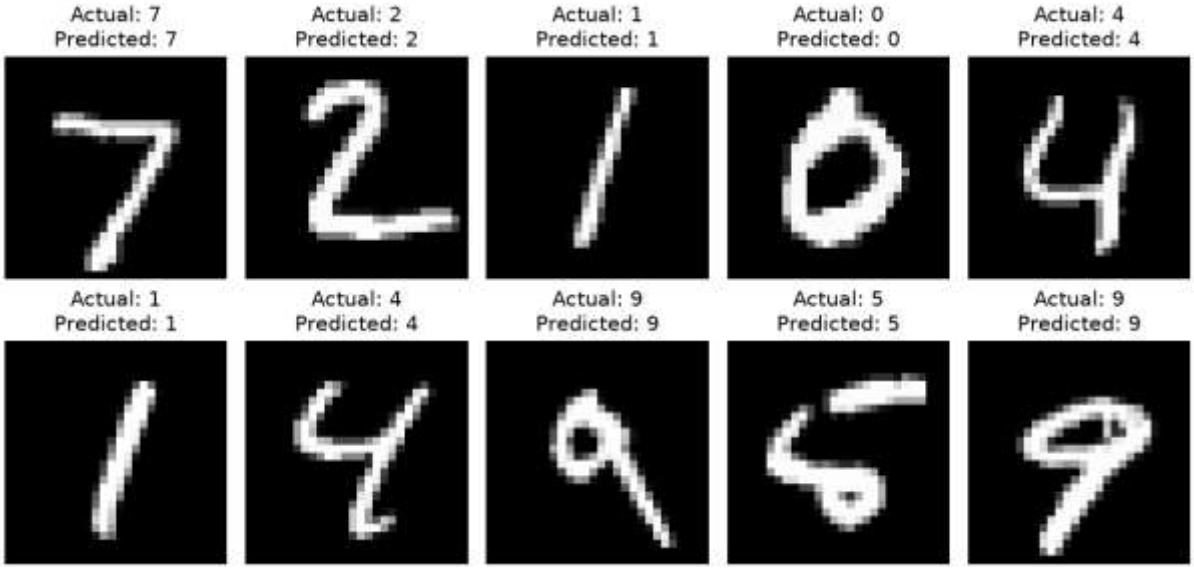
Hyperparameter tuning selects suitable learning rates, batch sizes, and model settings.

OUTPUT


Training and Validation Accuracy
Accuracy
Epoch
Training Accuracy
Validation Accuracy


Confusion Matrix
Actual Label
Predicted Label


Training and Validation Loss
Training Loss
Validation Loss
Loss
Epoch


Actual: 7
Predicted: 7
Actual: 2
Predicted: 2
Actual: 1
Predicted: 1
Actual: 0
Predicted: 0
Actual: 4
Predicted: 4
Actual: 1
Predicted: 1
Actual: 4
Predicted: 4
Actual: 9
Predicted: 9
Actual: 5
Predicted: 5
Actual: 9
Predicted: 9